

OCI Generative AI In This Application

Short implementation note

This application uses OCI Generative AI as part of the live runtime path. The OCI services are not attached around the edges. They are used directly for response generation, conversation state, document storage and retrieval, remote tool access, and comparison analysis.

AGNTCY is used for discovery and trust. OCI Generative AI is used for the actual AI work once the right agent is selected.

Implementation view



What this means in practice

The application first decides which kind of request it received. It then uses AGNTCY to find the right A2A agent and confirm that the agent is trusted. After that, the selected agent uses OCI Generative AI services to complete the task.

OCI services used right now

- Responses API is the main runtime model layer.
- Conversations are used for request session state.
- Files API and Vector Store are used for document storage and retrieval.
- Remote MCP is used for live web search through hosted SerpApi MCP.
- Code Interpreter, Embeddings, and Guardrails are used in the comparison flow.

Responses API

Used as the main path for model calls across the app.

Vector Store

Used for document memory, retrieval, and document question answering.

Remote MCP

Used by the web search agent to fetch live web results.

Conversations

Used in CLI and hosted entry points so requests can keep a conversation id.

Comparison support

Code Interpreter + Embeddings + Guardrails are used in the comparison agent.

Where Each OCI Service Fits

Focused technical mapping

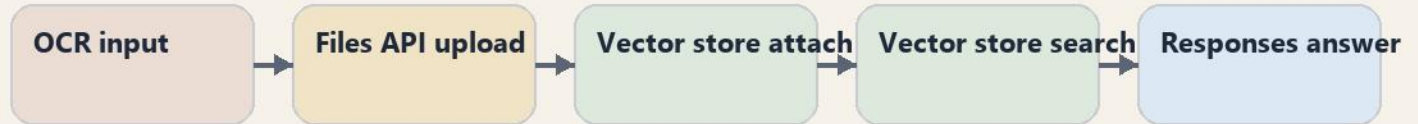
OCI service	Role	How it is used	Main code
Responses API	Main response generation layer	Used to generate grounded answers, comparison outputs, remote MCP calls, and tool-based model responses.	responses_client.py, orchestrator_agent.py, comparison_agent.py, web_search_agent.py
Conversations	Session continuity	Creates conversation ids for CLI and hosted requests so the runtime can keep request context.	main.py, hosted_app.py, responses_client.py
Files API	Document upload path	OCR text is converted into a temporary text file and uploaded before vector-store attachment.	retrieval_client.py
Vector Store	Document retrieval layer	The uploaded file is attached to a shared vector store and later searched for matching chunks.	retrieval_client.py, document_ingest_agent.py, document_query_agent.py
Remote MCP	Live tool integration	The web search agent uses OCI remote MCP to call hosted SerpApi MCP without a local web tool server.	web_search_agent.py, responses_client.py
Code Interpreter	Comparison helper	Used to calculate simple metrics over the two inputs before final structured comparison is produced.	comparison_agent.py, responses_client.py
Embeddings	Similarity signal	Used in the comparison flow to estimate how close the two inputs are before summarizing changes.	embeddings_client.py, comparison_agent.py
Guardrails	Safety check in comparison path	Applied around comparison input and output so that flow has an additional validation layer.	guardrails_client.py, responses_client.py, comparison_agent.py

Current OCI-Based Workflow

How the runtime uses OCI services today

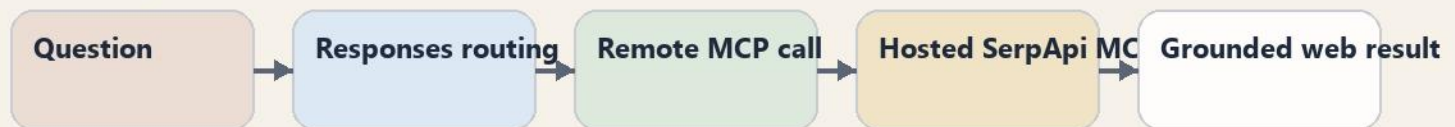
The three flows below show where OCI Generative AI is involved in the current runtime. The descriptions are written from the OCI usage point of view rather than from the full app architecture point of view.

1. Document question flow



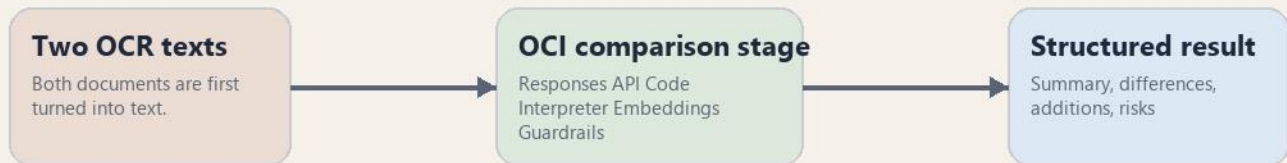
- OCR text is first extracted from the document.
- That text is uploaded into OCI through the Files API.
- The uploaded file is attached to the shared vector store.
- When the user asks a question, the document query agent searches the same vector store.
- The grounded answer is returned from retrieved chunks.

2. General question flow



- The web search agent uses OCI remote MCP instead of a local web tool server.
- OCI passes the request to the hosted SerpApi MCP endpoint.
- The returned grounded snippet is sent back as the answer for pure web questions.

3. Comparison flow



- OCI Responses produces the structured comparison output.
- OCI Code Interpreter is used for simple supporting metrics.
- OCI Embeddings add a similarity signal.
- OCI Guardrails are used around the comparison path.

Summary

The current design keeps OCI Generative AI close to the main runtime. Document retrieval, web lookup, comparison support, and conversation handling all depend on OCI services in the request path.